Synchronization

Victor Eijkhout

2026 PCSE

Need for synchronization

- The loop and sections directives do not specify an ordering, sometimes you want to force an ordering.
- Barriers: global synchronization.
- Critical sections: only one process can execute a statement this prevents race conditions.
- Locks: protect data items from being accessed.

Barriers

- Every workshare construct has an implicit barrier:

```
#pragma omp parallel
{
  #pragma omp for
    for ( .. i .. )
      x[i] = ...
  #pragma omp for
    for ( .. i .. )
      y[i] = .. x[i] .. x[i+1] .. x[i-1] ...
}
```

  First loop is completely finished before second.

- Explicit barrier:

```
#pragma omp parallel
{
  x = f();
#pragma omp barrier
  .... x ...
}
```

Nowait

```
1 #pragma omp parallel
2 {
3   #pragma omp for nowait
4   for ( int i=0; i<N; ++i )
5     x[i] = // something
6   #pragma omp for
7   for ( int i=0; i<N; ++i )
8     y[i] = ... x[i] ...
9 }
```

Needed:

- In the same parallel region (why?)
- Same number of iterations
- Same schedule

Critical sections

- Critical section: One update at a time.

```
#pragma omp parallel
{
  double x = f();
#pragma omp critical
  global_update(x);
}
```

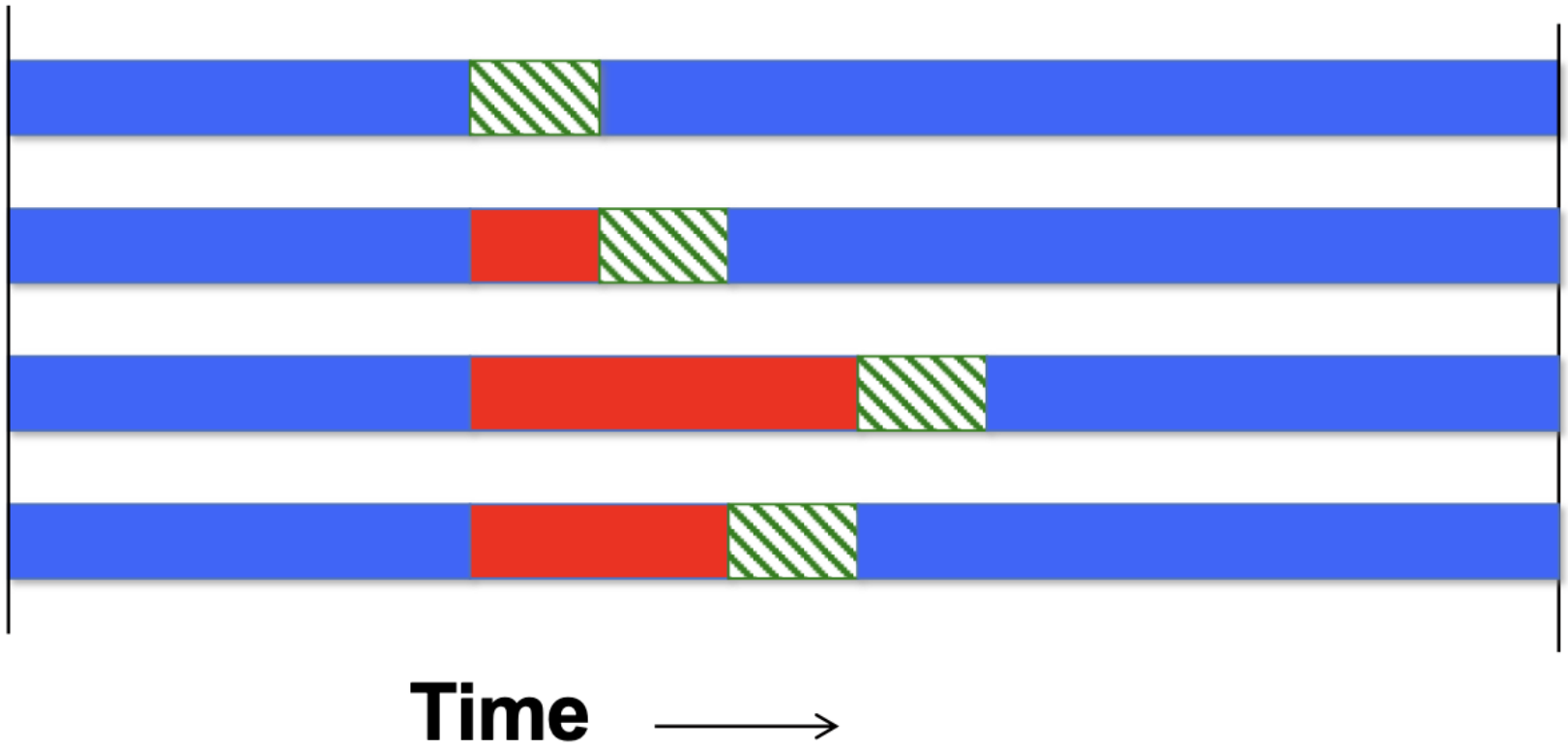
- *atomic* : special case for simple operations, possible hardware support

```
#pragma omp atomic
  t += x;
```

Warning

- Critical sections are not cheap! The operating system takes thousands of cycles to coordinate the threads.
- Use only if minor amount of work.
- Do not use if a reduction suffices.
- Name your critical sections.
- Explore locks if there may not be a data conflict.

Critical sections and idle time



In effect, the execution becomes sequential!

Do not misuse critical sections

```
1 #pragma omp parallel for
2 for ( /* i in whatever */ ) {
3   #pragma omp critical
4   s += /* expression with i */
5 }
```

- Use reduction if appropriate
- Only use if the enough other work in the iteration

Locks

- Critical sections are coarse:
  they dictate exclusive acces to a *statement*
- Suppose you update a big table
  updates to non-conflicting locations should be allowed
- Locks protect a single data item.

Lock create and destoy

- Variable to hold lock
- Create/destroy actual lock

```
1 // lock.c
2 omp_lock_t the_lock;
3 omp_init_lock( &the_lock );
4 omp_destroy_lock( &the_lock );
```

Use of lock

- Same thread sets/unsets lock
- While lock is set, other threads can not pass

```
1 #pragma omp parallel
2   {
3     omp_set_lock( &the_lock );
4     sum += omp_get_thread_num();
5     omp_unset_lock( &the_lock );
6   }
```

Exercise: histogram

Basic idea:

```
for ( i /* lots of cases */ )
  bin[ property(i) ]++;
```

Without a specific application, we let the bin number be random:

```
// histogramwrong.c
for ( long int experiment=0; experiment<nexperiments; ++experiment ) {
  int bin;
  for ( int r=0; r<1000; r++ ) bin = rand_r(&seed)%nbins;
  bins[bin]++;
}
```

Histogram 2

- Make this omp parallel
- Observe that the result is not correct.
- Experiment with number of threads and bins.

Histogram 3

- Solve with histogram
- Experiment with number of threads and bins.

Histogram 3

- Solve with locks
  (How many locks are needed?)
- Experiment with number of threads and bins.